

Lab Observation Report: Task 31

Bank Account Management System using Inheritance, Method Overriding & Inter-Account Transfers

Course / Topic:	Java Programming – OOP Inheritance & Dynamic Polymorphism	Task ID:	Observation Task 31
Language:	Java (JDK 8 or higher)	Execution Mode:	Console Application (CLI)

1. Objective

The primary objective of this program is to model a banking system hierarchy using Object-Oriented Programming (OOP) principles in Java. The application demonstrates core structural and behavioral concepts, specifically single/ hierarchical inheritance, runtime polymorphism (method overriding), and object interaction through inter-account money transfers.

- To design a base class Account encapsulating common account attributes (account number, holder name, balance) and essential banking operations (deposit, withdraw, interest calculation, transfer, display).
- To extend the base class into specialized account types: SavingsAccount (incorporating interest calculation) and CurrentAccount (supporting overdraft facilities and maintenance fees).
- To demonstrate method overriding for withdraw() and calinterest() to enforce custom business rules per account type.
- To execute inter-account money transfers using object references.

2. Systems Requirement

Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	1.6 GHz Dual-Core Processor	2.0 GHz Quad-Core Intel i5 / AMD Ryzen 5 or higher
RAM	2 GB	8 GB or higher
Storage Space	500 MB free hard disk space	1 GB SSD space
Display / Input	Standard Display (1024x768), Keyboard & Mouse	Standard Full HD Display, Keyboard & Mouse

Software Requirements

Software Tool	Specification
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
Java Development Kit (JDK)	JDK 8 / JDK 11 / JDK 17 / JDK 21 installed with JAVA_HOME configured

IDE / Code Editor	VS Code, Eclipse, IntelliJ IDEA, or command-line Terminal
Compiler / Runtime	<code>javac</code> (Java Compiler) and <code>java</code> (Java Virtual Machine)

3. Algorithm & Logic Flow

Step 1: Account Instantiation

In the main driver class `ObservationTask31`, instantiate two account objects:

- `CurrentAccount ca` with account number "UB 24311", holder "kumar", balance `4500.0`, and overdraft limit `2000.0`.
- `SavingsAccount sa` with account number "SB 24533", holder "hemanth", balance `5000.0`, and interest rate `0.04` (4%).

Step 2: Display Initial State

Invoke `show()` on both `ca` and `sa` to display initial account details (Account Number, Holder Name, Balance).

Step 3: Execute Current Account Transactions

- **Deposit:** Call `ca.deposit(2000)`. Validate amount > 0 , update balance ($\$4500 + 2000 = \6500).
- **Withdraw:** Call `ca.withdraw(2500)`. Check against balance + overdraft limit ($\$6500 + 2000 = \8500). Valid withdrawal updates balance ($\$6500 - 2500 = \4000).
- **Interest / Maintenance:** Call `ca.calinterest()`. Deduct monthly maintenance charge of `50.0` ($\$4000 - 50 = \3950).

Step 4: Execute Savings Account Transactions

- **Withdraw:** Call `sa.withdraw(3000)`. Validate against available balance ($\$5000$). Valid withdrawal updates balance ($\$5000 - 3000 = \2000).
- **Calculate Interest:** Call `sa.calinterest()`. Compute interest ($\$2000 \times 0.04 = \80.0) and credit to balance ($\$2000 + 80 = \2080).

Step 5: Execute Inter-Account Transfer

Call `sa.transfer(ca, 1000)` to transfer `1000.0` from `sa` to `ca`:

- Invoke `sa.withdraw(1000)`: Updates `sa` balance ($\$2080 - 1000 = \1080).
- Invoke `ca.deposit(1000)`: Updates `ca` balance ($\$3950 + 1000 = \4950).

Step 6: Display Final State

Invoke `ca.show()` and `sa.show()` to display the updated state of both accounts.

4. UML Diagrams

4.1 Class Hierarchy Diagram

```
+-----+ | Account
| +-----+ | -
| accountnumber: String | | - accountholder: String | | + balance: double |
+-----+ | +
| Account(accountnumber: String, accountholder: String, balance: double) | | +
| getaccountnumber(): String | | + getaccountholder(): String | | + getbalance(): double | | ~
| deposit(amount: double): void | | ~ withdraw(amount: double): void | | ~ calinterest(): void |
| ~ transfer(fixedAccount: Account, amount: double): void | | ~ show(): void |
+-----+ ^ |
+-----+ | | +-----+
+-----+ | SavingsAccount | | CurrentAccount |
+-----+ +-----+ | - interestrate:
| double | | - overdraftlimit: double | +-----+
+-----+ | + SavingsAccount(...) | | + CurrentAccount(...) | | ~
| withdraw(amount: double): void | | ~ withdraw(amount: double): void | | ~ calinterest(): void |
| ~ calinterest(): void | +-----+
+-----+
```

5. Source Code Overview

```
class Account {
    private String accountnumber;
    private String accountholder;
    public double balance;

    // Constructor
    public Account(String accountnumber, String accountholder, double balance) {
        this.accountnumber = accountnumber;
        this.accountholder = accountholder;
        this.balance = balance;
    }

    // Getters
    public String getaccountnumber() {
        return accountnumber;
    }

    public String getaccountholder() {
        return accountholder;
    }

    public double getbalance() {
        return balance;
    }

    // Deposit
    void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("Deposit amount must be greater than zero");
        } else {
            balance = balance + amount;
            System.out.println("Deposited amount is " + amount);
        }
    }
}
```

```

    }
}

// Withdraw
void withdraw(double amount) {
    System.out.println("Withdrawal operation");
}

// Calculate interest
void calinterest() {
    System.out.println("Interest calculation");
}

// Transfer
void transfer(Account fixedAccount, double amount) {
    System.out.println("Transfer of " + amount + " from "
        + accountnumber + " to "
        + fixedAccount.getaccountnumber());

    this.withdraw(amount);
    fixedAccount.deposit(amount);
}

// Display
void show() {
    System.out.println("ACCOUNT NUMBER IS " + accountnumber);
    System.out.println("ACCOUNT HOLDER IS " + accountholder);
    System.out.println("BALANCE IS " + balance);
}
}

// Savings Account
class SavingsAccount extends Account {
    private double interestrate;

    public SavingsAccount(String accountnumber, String accountholder,
        double balance, double interestrate) {

        super(accountnumber, accountholder, balance);
        this.interestrate = interestrate;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Withdraw amount is not possible");
        } else if (amount > balance) {
            System.out.println("Insufficient balance");
        } else {
            balance = balance - amount;
            System.out.println("Withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double interest = balance * interestrate;
        balance = balance + interest;

        System.out.println("Interest is credited: " + interest);
    }
}

```

```

}

// Current Account
class CurrentAccount extends Account {
    private double overdraftlimit;

    public CurrentAccount(String accountnumber, String accountholder,
                           double balance, double overdraftlimit) {

        super(accountnumber, accountholder, balance);
        this.overdraftlimit = overdraftlimit;
    }

    @Override
    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Withdraw amount is not possible");
        } else if (amount > balance + overdraftlimit) {
            System.out.println("Insufficient balance and overdraft limit exceeded");
        } else {
            balance = balance - amount;
            System.out.println("Withdraw is " + amount);
        }
    }

    @Override
    void calinterest() {
        double charge = 50.0;
        balance = balance - charge;

        System.out.println("Maintenance charge deducted: " + charge);
    }
}

// Main class
public class ObservationTask31 {
    public static void main(String[] args) {

        CurrentAccount ca =
            new CurrentAccount("UB 24311", "kumar", 4500, 2000);

        SavingsAccount sa =
            new SavingsAccount("SB 24533", "hemanth", 5000, 0.04);

        System.out.println("INITIAL ACCOUNT DETAILS");

        ca.show();

        System.out.println();

        sa.show();

        System.out.println();
        System.out.println("TRANSACTION DETAILS");

        ca.deposit(2000);

        ca.withdraw(2500);

        ca.calinterest();
    }
}

```

```

        sa.withdraw(3000);

        sa.calinterest();

        sa.transfer(ca, 1000);

        System.out.println();
        System.out.println("FINAL ACCOUNT DETAILS");

        ca.show();

        System.out.println();

        sa.show();
    }
}

```

6. Compilation & Execution Steps

Step 1: Save Source Code

Save the source code into a file named `ObservationTask31.java` (must strictly match the public main class name).

Step 2: Open Command Prompt / Terminal

Navigate to the folder containing your `ObservationTask31.java` file:

```
cd path/to/your/java/project
```

Step 3: Compile Java Program

Run the Java compiler:

```
javac ObservationTask31.java
```

Upon successful compilation, four bytecode files are generated: `Account.class`, `SavingsAccount.class`, `CurrentAccount.class`, and `ObservationTask31.class`.

Step 4: Execute Program

Launch the Java Virtual Machine (JVM) to execute the bytecode:

```
java ObservationTask31
```

7. Program Output

Below is the exact console output produced when executing the program:

```

INITIAL ACCOUNT DETAILS
ACCOUNT NUMBER IS UB 24311

```

ACCOUNT HOLDER IS kumar
BALANCE IS 4500.0

ACCOUNT NUMBER IS SB 24533
ACCOUNT HOLDER IS hemanth
BALANCE IS 5000.0

TRANSACTION DETAILS

Deposited amount is 2000.0
Withdraw is 2500.0
Maintenance charge deducted: 50.0
Withdraw is 3000.0
Interest is credited: 80.0
Transfer of 1000.0 from SB 24533 to UB 24311
Withdraw is 1000.0
Deposited amount is 1000.0

FINAL ACCOUNT DETAILS

ACCOUNT NUMBER IS UB 24311
ACCOUNT HOLDER IS kumar
BALANCE IS 4950.0

ACCOUNT NUMBER IS SB 24533
ACCOUNT HOLDER IS hemanth
BALANCE IS 1080.0